

Chapter-2: Constants, Variables and Data Types

1. What is token? [2022]

Answer: A token is the smallest individual element in a program, such as a keyword, identifier, operator, or constant.

2. Describe different types of tokens. [2022]

Answer: In the context of C programming, a token refers to a sequence of characters that represent a single unit of meaning. We can define the token as the smallest individual element in C. Tokens are like parts of speech in English grammar. I eat rice. Here, 'I' is pronoun, 'eat' is verb, and 'rice' is noun. Tokens are the basic building blocks of a program, and they can be classified into different types such as keywords, identifiers, constants, strings, special symbols, and operators.

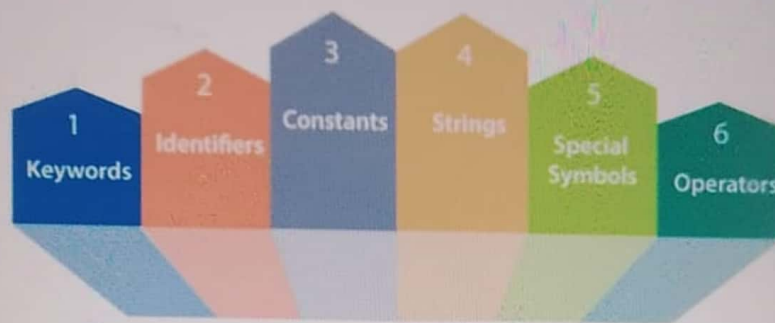


Figure: Classification of Token in C

- **Keywords:** Keywords are reserved words in C that have specific meanings and cannot be used as identifiers. There are 32 keywords in C.
Example: int, float, if, while.
- **Identifiers:** Identifiers are user-defined names for variables, functions, arrays, etc. They must follow certain rules, such as starting with a letter and consisting of letters, digits, or underscores. The first digit of an identifier must be a letter or underscore.
Example: Age, area, PI, sum, _name.
- **Constants:** Constants are fixed values that do not change during program execution. It includes integer constants, floating-point constants, and character constants. It can be declared in two ways such as using const keyword and #define directive.
Example: `const float height = 5.8;`
`#define PI 3.1416`
- **Strings:** Strings are sequences of characters enclosed in double quotes. They are used to represent textual data.
Example: `char greeting[100] = "Welcome to CSE!";`
- **Special Symbols:** Special symbols include punctuation and other symbols used for specific purposes in C.
Examples: braces '{}', parentheses '()', semicolons ';', commas ','.

- **Operators:** In programming, an operator is a symbol or keyword that represents a specific operation or action to be performed on one or more operands.

Example: Arithmetic operators (+, -, *, /, %), assignment operator (=), sizeof operator.

Mastering the types of tokens is essential for understanding the syntax of C programs. A solid grasp of these elements lays the foundation for efficient and logical coding practices.

3. Define token, keyword, identifier, variable and constant. [2019, 2017]

Answer: Definition of token, keyword, identifier, variable and constant are given below:

- **Token:** A token is the smallest individual element in a program, such as a keyword, identifier, operator, or constant.
- **Keyword:** Keywords are reserved words in C that have specific meanings and cannot be used as identifiers. There are 32 keywords in C.
Example: int, float, if, while.
- **Identifier:** Identifiers are user-defined names for variables, functions, arrays, etc. They must follow certain rules, such as starting with a letter and consisting of letters, digits, or underscores. The first digit of an identifier must be a letter or underscore.
Example: Age, area, PI, sum, _name.
- **Variable:** A variable is a named memory location used to store data that can be modified during program execution.
Example:

```
int age;
float temperature;
char letter;
```
- **Constant:** Constants are fixed values that do not change during program execution. It includes integer constants, floating-point constants, and character constants. It can be declared in two ways such as using const keyword and #define directive.
Example:

```
const float height = 5.8;
#define PI 3.1416
```

4. What are keywords and identifiers? Define with proper examples. [2023]

Answer: In C programming, keywords and identifiers are fundamental building blocks of any program. Definition of keywords and identifiers with proper examples are given below:

- **Keyword:** Keywords are reserved words in C that have specific meanings and cannot be used as identifiers. There are 32 keywords in C.
Example: int, float, if, while.
- **Identifier:** Identifiers are user-defined names for variables, functions, arrays, etc. They must follow certain rules, such as starting with a letter and consisting of letters, digits, or underscores. The first digit of an identifier must be a letter or underscore.
Example: Age, area, PI, sum, _name.

5. Find the invalid identifiers from the following and explain it: [2016]

(i) for; (ii) -abc; (iii) Maximum; (iv) CSE Dept.

Answer: The explanation is given below:

- (i) **for:** for is an invalid identifier because it is a reserved keyword in C used for creating loops, and identifiers cannot have the same names as keywords.
- (ii) **-abc:** -abc is an invalid identifier because identifiers cannot start with a hyphen (-) symbol. Valid identifiers must start with an alphabet (A-Z or a-z) or an underscore (_).
- (iii) **Maximum:** Maximum is a valid identifier as it starts with an alphabet (M) and does not contain any invalid characters.
- (iv) **CSE Dept.:** CSE Dept. is an invalid identifier because identifiers cannot contain spaces or special characters like a dot (.). Valid identifiers must consist of only letters, digits, and underscores, and cannot start with a digit. To make it a valid identifier, spaces and special characters should be removed (e.g., CSE_Dept).

6. Define constant. [2017]

Answer: In C, a constant is a fixed value that does not change during program execution. Constants are used to represent data that remains the same throughout the program and cannot be modified after initialization.

There are two types of constants in C:

- **Numeric Constants:** Numeric constants represent numerical values such as integers, and floating-point numbers.
Example: `const int ID = 230200;`
- **Symbolic Constants:** Symbolic constants are also known as "named constants" or "macros." They are represented by identifiers and are defined using the `#define` preprocessor directive.
Example: `#define PI 3.1416`

7. What is variable? [2023, 2022, 2020, 2017]

Answer: In C, a variable is a named memory location used to store data that can be modified during program execution. Variables allow programmers to manipulate data and perform computations dynamically based on the values they hold.

In C, variables must be declared with a specific data type before they can be used. This data type determines the kind of data the variable can hold and the operations that can be performed on it.

Example: `int ID;` // Here, ID is an integer type variable.

`float AGE;` // Here, AGE is a floating point type variable.

8. What are the rules for declaring variables? [2018]

Answer: In C, a variable is a named memory location used to store data that can be modified during program execution. There are specific rules for declaring variables:

- **Valid Characters:** Variable names can only contain letters (both uppercase and lowercase), digits, and underscores (_). They must start with a letter or an underscore and cannot start with a digit.
- **No Spaces or Special Characters:** Variable names cannot contain spaces or special characters (e.g., @, !, \$, etc.).
- **Keywords:** Variable names cannot be the same as C keywords, as keywords have predefined meanings in the C language and cannot be used as identifiers.
- **Case Sensitivity:** C is case-sensitive, meaning uppercase and lowercase letters are treated as different characters. So, age and Age would be considered as two separate variables.
- **Length Limit:** Variable names can be of any length, but only the first 31 characters are significant. It is good practice to use descriptive and meaningful names that are not too long.

Example of valid variable declarations:

```
#include <stdio.h>

int main() {
    int ID1;
    int ID2;
    float average_score;
    char _initial;

    return 0;
}
```

In this example, we have declared four variables: ID1, ID2, average_score, and _initial. The variable names adhere to the rules:

- ✓ They only contain valid characters (letters, digits, and underscores).
- ✓ They don't contain any spaces or special characters.
- ✓ They don't conflict with C keywords.
- ✓ They start with a letter or an underscore (not a digit).

Following these rules when declaring variables ensures that the program is valid and adheres to the syntax and conventions of the C programming language. Meaningful variable names also improve code readability and maintainability.

9. How can you define and initialize a variable in C program? [2023, 2020, 2017]

Answer: In C, variable must be defined or declared and initialize with a value before being used in the program. We can define and initialize a variable in the following way:

▪ **Variable Declaration:** Variable can be declared as follows:

- ✓ Syntax: `data_type variable_name;`

Here,

`data_type`: The data type of the variable, such as `int`, `float`, `char`, etc.

`variable_name`: The name of the variable, adhering to the rules for variable naming.

- ✓ Example: `int ID;`

`float height;`

Here, `ID` and `height` are two variables with `int` and `float` data types respectively.

- ✓ Once a variable is declared, it can be assigned a value using an assignment operator, and its value can be used in expressions and statements throughout the program.

▪ **Variable Declaration and Initialization:** Variable can be declared and initialized at the same time as follows:

- ✓ Syntax: `data_type variable_name = initial_value;`

Here,

`data_type`: The data type of the variable, such as `int`, `float`, `char`, etc.

`variable_name`: The name of the variable, adhering to the rules for variable naming.

`initial_value`: The initial value assigned to the variable at the time of declaration.

- ✓ Example: `int age = 30;`

`float weight = 65.5;`

Here, `age` and `weight` are two variables with `int` and `float` data types with value of 30 and 65.5 respectively.

A simple C program to demonstrate variable declaration and initialization is given below:

```
#include <stdio.h>

int main() {
    // Variable declaration
    int ID;
    float height;

    // Variable declaration and initialization
    int age = 30;
    float weight = 65.5;

    return 0;
}
```

Variable initialization provides an initial value to the variable, making sure it has a predictable value before being used in the program. If we don't provide an initial value during declaration, the variable will have an undefined value, and using it before initialization may lead to unexpected behavior. Initializing variables is a good practice to ensure consistent behavior and avoid potential bugs in our C programs.

10. What is a variable and what is meant by the "value" of a variable? [2016]

Answer: In computer programming, a variable is a named memory location used to store data. Variables are an essential concept in programming as they allow programmers to manipulate and work with data dynamically during the program's execution.

Each variable has a specific data type that determines the kind of data it can hold (e.g., integer, floating-point number, character, etc.). The data type also determines the size of memory allocated to the variable to store the value.

"Value" of a Variable: Value of variable means the data stored in that variable at a particular point in the program's execution on which we apply the mathematical operation. A variable value can be number, digit, time, amount of money, height, weight, quality, quantity, etc., or anything for measurement or calculation. A variable value may be data of an inquiry, from which we can find information. When a variable is declared and initialized, it is assigned an initial value, which can be changed or updated during the program's execution.

- **Variable Declaration and Initialization:** Variable can be declared and initialized at the same time as follows:

- ✓ Syntax: `data_type variable_name = initial_value;`

Here,

`data_type`: The data type of the variable, such as `int`, `float`, `char`, etc.

`variable_name`: The name of the variable, adhering to the rules for variable naming.

`initial_value`: The initial value assigned to the variable at the time of declaration.

- ✓ Example: `int age = 25;`

`float weight = 65.5;`

Here, `age` and `weight` are two variables with `int` and `float` data types with value of 25 and 65.5 respectively. Now, if we write `weight = 70.00;` in the next statement, then the updated value of the variable `weight` will be 70.00.

Now, consider the following C code:

```
#include <stdio.h>

int main() {
    int age = 25;
    printf("Age: %d\n", age);

    age = 30;

    printf("Updated Age: %d\n", age);
    return 0;
}
```

In this example, `age` is a variable of type `int`. It starts with an initial value of 25, and later, the value is updated to 30. The "value" of the `age` variable changes over time as the program assigns new values to it.

11. Distinguish between local variable and global variable. [2022, 2020, 2018, 2016]

Answer: Differences between local variable and global variable are given below:

Aspect	Local Variable	Global Variable
Definition	A local variable is a variable that is declared within a specific block or function and has a limited scope within that block or function.	A global variable is a variable that is declared outside of any function and is accessible throughout the entire program.
Scope	Limited to the block or function where it is declared.	Visible and accessible throughout the entire program.
Storage Duration	Created when the function is called, destroyed when the function exits.	Created at the start of the program, destroyed when the program ends.
Lifetime	Exists only while the function is active.	Exists throughout the entire execution of the program.
Access	Accessed within the block or function where it is declared.	Accessed from any part of the program.
Example	<pre>int main(){ int x = 10; //Local variable printf("Output: %d", x); return 0; }</pre>	<pre>int TAX = 20; //Global variable int main(){ printf("Output: %d", TAX); return 0; }</pre>

12. Describe with example the four basic data types used in a C program. [2022, 2020, 2018, 2016]

Answer: A data type is a classification or category that specifies the type of data that a variable can hold or represent. It determines the size of memory allocated to the variable and the operations that can be performed on the data. C provides several built-in data types that programmers can use to declare variables and work with different kinds of data.

In C, the four basic data types are int, float, char, and double. Each data type is used to represent different types of values, and they are fundamental building blocks for storing and manipulating data in a C program. Here's a description of each data type with an example:

- **int:** The int data type is used to store integer values, which are whole numbers without decimal points. "int" keyword is used to refer integer data type. The storage size of int data type is 2 or 4 bytes. It varies depend upon the processor in the CPU that we use. We can't store decimal values using int data type. If we use int data type to store decimal values, decimal values will be truncated and we will get only whole number. The format specifier for int data type is usually "%d" or "%i".
Example: `int age = 30;`
- **float:** The float data type is used to store floating-point values, which are numbers with decimal points. "float" keyword is used to refer floating point data type. Storage size of

float data type is 4 bytes. We can use up-to 6 digits after decimal using float data type. For example, 10.456789 can be stored in a variable using float data type. The format specifier for float data type is "%f".

Example: float height = 5.8;

- **char:** The char data type is used to store single characters, enclosed within single quotes (' '). Character data type allows a variable to store only one character at a time. "char" keyword is used to refer character data type. Storage size of char data type is 1 byte. For example, 'A' can be stored using char data type. The format specifier for char data type is "%c".

Example: char grade = 'A';

- **double:** The double data type is used to store double-precision floating-point values, which are similar to float but with higher precision. "double" keyword is used to refer double data type. Storage size of double data type is 8 bytes. The format specifier for double data type is usually "%lf".

Example: double savings = 500000000.00;

Example: A simple C program to demonstrate all of these four basic data types is given below:

```
#include <stdio.h>

int main() {
    int age = 30;
    float weight = 65.5;
    char grade = 'A';
    double pi = 3.14159;

    printf("Age: %d\n", age);
    printf("Weight: %.2f\n", weight);
    printf("Grade: %c\n", grade);
    printf("Pi: %.5lf\n", pi);

    return 0;
}
```

13. How could we extend the range of values they (data types) represent? [2022, 2016]

Answer: The amount of memory space to be allocated for a variable is derived by modifiers. Modifiers are prefixed with basic data types to modify (either increase or decrease) the amount of storage space allocated to a variable. For example, storage space for int data type is 4 bytes for 32 bit processor. We can increase the range by using long int which is 8 bytes. To extend the range of values that data types can represent in C, we can use the following methods:

- **Use Larger Data Types:** We can choose data types that have a larger size, allowing them to represent a broader range of values. For example, if the int data type is not sufficient to

represent the range of values we need, we can use long int or long long int which have larger sizes.

Example: long long int bigNumber;

- **Use Unsigned Data Types:** If the range of values we need to represent is entirely positive or non-negative, consider using unsigned data types (e.g., unsigned int, unsigned long) as they allow us to use the entire range of the data type for positive values.

Example: unsigned int population;

- **Use Floating-point Data Types:** If we need to represent numbers with decimal points or a wider range of fractional values, we can use floating-point data types like float, double, or long double.

Example: long double PI;

It's essential to consider the trade-offs between size and memory usage when selecting data types. Using larger data types may increase memory consumption, so choose the data type that best suits the range and precision requirements of our program.

Example: In the example given below, we use the long long int data type to store a large value that cannot be represented using a regular int. The LL suffix indicates that the number is of type long long int.

```
#include <stdio.h>

int main() {
    long long int bigNumber = 9999999999999999LL;

    printf("Big Number: %lld\n", bigNumber);

    return 0;
}
```

14. What is symbolic constant? [2023, 2021, 2019]

Answer: A symbolic constant, also known as a "named constant" or "macro constant," is an identifier that represents a fixed value in a C program. It is defined using the #define preprocessor directive and allows us to give a meaningful name to a constant value, making the code more readable and maintainable.

Syntax: #define constant_name value

Example: #define PI 3.14159

15. Write down the rules for declaring symbolic constant. [2021, 2019]

Answer: A symbolic constant, also known as a "named constant" or "macro constant," is an identifier that represents a fixed value in a C program. It is defined using the #define preprocessor directive and allows us to give a meaningful name to a constant value, making the code more

readable and maintainable. They are usually defined at the beginning of the program. The symbolic constants may then appear later in the program in place of the numeric constants, character constants, or a string. Symbolic constants are evaluated by the preprocessor. Unlike a variable, after a symbolic constant is initialized, its value can't be changed.

Syntax: `#define constant_name value`

Example: `#define PI 3.14159`

`#define NAME "MMR"`

Rules for Declaring Symbolic Constants:

- '#' sign must be the first character.
- The `#define` directive is used to define a symbolic constant.
- No blank space between pound sign and the keyword `define` is permitted.
- At least one blank space between `#define` and symbolic name and between the symbolic name and the constant value.
- Symbolic names are not declared for data types, it's data type depends on the type of constant value.
- The name of the symbolic constant should follow the rules for variable naming in C.
- Assignment operator (`=`) is not allowed for storing values.
- There should be no semicolon (`;`) at the end of the `#define` statement.
- The symbolic constant is not a variable and cannot be changed during program execution. That means, after definition, the symbolic name should not be assigned any other value within the program using assignment statement.
- Symbolic constants are case-sensitive.
- The value of a symbolic constant can be any valid C expression.

Example: In the example given below, we have defined a symbolic constant `PI`. This symbolic constant represents fixed value: 3.14159, and is used throughout the program in place of its actual value.

```
#include <stdio.h>

#define PI 3.14159

int main() {
    const float radius = 5.0;

    float area = PI * radius * radius;

    printf("Area of the circle: %.2f\n", area);

    return 0;
}
```



```

float area = PI * radius * radius;

printf("Area of the circle: %.2f\n", area);

return 0;

```

16. How does variable and symbolic constant differ? Give example. [2023]

Answer: Differences between variable and symbolic constant with example are given below:

Aspect	Variable	Symbolic Constant
Definition	Variable is a storage location in memory with a name that holds data, which can change during program execution.	Symbolic constant is a named constant whose value remains fixed throughout program execution.
Declaration	Declared using data types like int, float, etc., e.g.: int x = 5; float height = 5.8;	Declared using <code>#define</code> with no data types, no assignment operator and no semicolon at the end, e.g.: <code>#define PI 3.1416</code>
Value Change	The value of a variable can be modified during the program execution. For example: int main(){ int x = 5; x = 30; //This is valid return 0; }	The value of a symbolic constant cannot be reassigned or changed after it is defined. For example: <code>#define PI 3.1416</code> int main(){ float PI = 3.1416; //This is invalid return 0; }
Storage	Allocated memory in RAM for its value.	Does not occupy memory; it is replaced directly during compilation.
Usage	Used for storing data that may vary during runtime.	Used for defining fixed values such as constants or configuration settings.
Example	int main(){ int x = 5; x = 30; printf("Output: %d", x); return 0; } Output: 30	<code>#define PI 3.1416</code> int main(){ printf("Output: %.4f", PI); return 0; } Output: 3.1416

NB: This notebook is created using several reference books of C, ChatGPT, and other external sources. Please let me know if you find any mistakes. Thank you.